Classical ML Refresher Through a Quantum Lens

EVA: Quantum Machine Learning – Lecture 4

Pavel Sulimov

Spring 2026

ZHAW Zürcher Hochschule für Angewandte Wissenschaften

Roadmap

1. Linear models and why separability matters
2. Explicit feature maps, Gram matrices, and kernel geometry
3. Why this language transfers naturally to quantum states

Linear models and decision functions

Supervised learning in matrix form

With data matrix $X \in \mathbb{R}^{N \times d}$, parameters $w \in \mathbb{R}^d$, and bias $b \in \mathbb{R}$:

$$z = Xw + b1, \qquad \hat{y} = g(z).$$

Typical choices:

- regression: $g(z) = z$
- binary classification: $g(z) = \sigma(z) = \frac{1}{1+e^{-z}}$

`lecture_04_demo.ipynb`, Demo 1: matrix-form linear/logistic baseline.

Logistic regression objective

For labels $y_i \in \{0, 1\}$ and probabilities $p_i = \sigma(w^\top x_i + b)$, the negative log-likelihood is

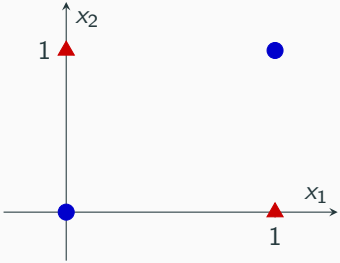
$$\mathcal{L}(w, b) = -\sum_{i=1}^{N} \left[y_i \log p_i + (1 - y_i) \log(1 - p_i) \right].$$

The key geometric fact is unchanged:

$$w^\top x + b = 0$$

is a hyperplane in the original feature space.

Why XOR breaks a linear model



The classes alternate across the square corners.

A linear score

$$s(x) = w_1 x_1 + w_2 x_2 + b$$

cannot make $(0, 0)$ and $(1, 1)$ share one sign while $(0, 1)$ and $(1, 0)$ share the opposite sign.

We need a richer representation, not just a different threshold.

Manual lifting for XOR

A useful explicit feature map is

$$\psi(x_1, x_2) = (x_1, x_2, x_1 x_2).$$

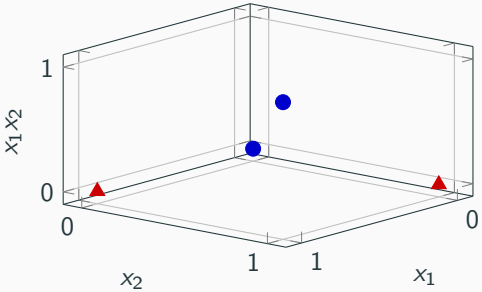
Input x	Lifted vector $\psi(x)$
$(0, 0)$	$(0, 0, 0)$
$(0, 1)$	$(0, 1, 0)$
$(1, 0)$	$(1, 0, 0)$
$(1, 1)$	$(1, 1, 1)$

This new coordinate $x_1 x_2$ captures an interaction that was missing in the original plane.

`lecture_04_demo.ipynb`, Demo 2: explicit XOR lifting.

XOR after lifting to 3D

$$\psi(x_1, x_2) = (x_1, x_2, x_1 x_2)$$



One separating hyperplane is

$$s = x_1 + x_2 - 2x_3 - \frac{1}{2}.$$

Checking the four points:

- $(0, 0, 0)$ and $(1, 1, 1)$ give $s = -0.5$
- $(0, 1, 0)$ and $(1, 0, 0)$ give $s = +0.5$

`lecture_04_demo.ipynb`, Demo 2: XOR lifting visualization.

Gram matrices and kernel geometry

From lifted vectors to a Gram matrix

Once we have feature vectors $\phi(x^{(i)})$, the Gram matrix is

$$G_{ij} = \langle \phi(x^{(i)}), \phi(x^{(j)}) \rangle.$$

For the explicit XOR lift $\psi(x) = (x_1, x_2, x_1 x_2)$:

$$\Psi = \begin{bmatrix} 0 & 0 & 0 \\ 0 & 1 & 0 \\ 1 & 0 & 0 \\ 1 & 1 & 1 \end{bmatrix}, \qquad G = \Psi\Psi^\top = \begin{bmatrix} 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 1 \\ 0 & 0 & 1 & 1 \\ 0 & 1 & 1 & 3 \end{bmatrix}.$$

Each entry stores a pairwise similarity in the lifted space.

`lecture_04_demo.ipynb`, Demo 3: explicit Gram matrix construction.

Computing Gram-matrix entries by hand

Using the lifted vectors:

$$\psi(0,1) = (0,1,0), \qquad \psi(1,1) = (1,1,1).$$

Then

$$G_{24} = \langle \psi(0,1), \psi(1,1) \rangle = 0 \cdot 1 + 1 \cdot 1 + 0 \cdot 1 = 1.$$

Also

$$G_{44} = \langle \psi(1,1), \psi(1,1) \rangle = 1^2 + 1^2 + 1^2 = 3.$$

So the diagonal entries are squared norms, while off-diagonal entries measure how aligned two feature vectors are.

Kernel trick: same idea, but implicit

A kernel is a function

$$K(x, y) = \langle \phi(x), \phi(y) \rangle$$

for some feature map ϕ, possibly without writing ϕ explicitly.

For the degree-2 polynomial kernel

$$K_{\text{poly}}(x, y) = (x^\top y + 1)^2,$$

one valid explicit feature map is

$$\phi_{\text{poly}}(x_1, x_2) = \left(1, \sqrt{2}x_1, \sqrt{2}x_2, x_1^2, \sqrt{2}x_1x_2, x_2^2\right).$$

Important: this is not the same map as $\psi(x_1, x_2) = (x_1, x_2, x_1x_2)$.

Polynomial kernel on XOR

Inputs:

$$x^{(1)} = (0,0),\ x^{(2)} = (0,1),\ x^{(3)} = (1,0),\ x^{(4)} = (1,1).$$

With

$$K_{\text{poly}}(x,y) = (x^\top y + 1)^2,$$

the Gram matrix is

$$K = \begin{bmatrix} 1 & 1 & 1 & 1 \\ 1 & 4 & 1 & 4 \\ 1 & 1 & 4 & 4 \\ 1 & 4 & 4 & 9 \end{bmatrix}.$$

Example:

$$K_{24} = ((0,1)\cdot(1,1)+1)^2 = (1+1)^2 = 4.$$

`lecture_04_demo.ipynb`, Demo 3: explicit map vs polynomial kernel.

Distance can be recovered from kernels

Inner products are enough to recover squared distances in feature space:

$$\|\phi(x) - \phi(y)\|^2 = K(x,x) + K(y,y) - 2K(x,y).$$

For the polynomial kernel and points

$$x = (0,1), \qquad y = (1,0),$$

we have

$$K(x,x) = 4, \qquad K(y,y) = 4, \qquad K(x,y) = 1.$$

Therefore

$$\|\phi_{\text{poly}}(x) - \phi_{\text{poly}}(y)\|^2 = 4 + 4 - 2 \cdot 1 = 6.$$

The kernel gives geometry without listing all feature coordinates explicitly.

`lecture_04_demo.ipynb`, Demo 4: recover distances from kernel values.

RBF kernel: similarity from distance directly

The radial basis function (RBF) kernel is

$$K_{\mathrm{RBF}}(x, y) = \exp\left(-\gamma \|x - y\|^2\right).$$

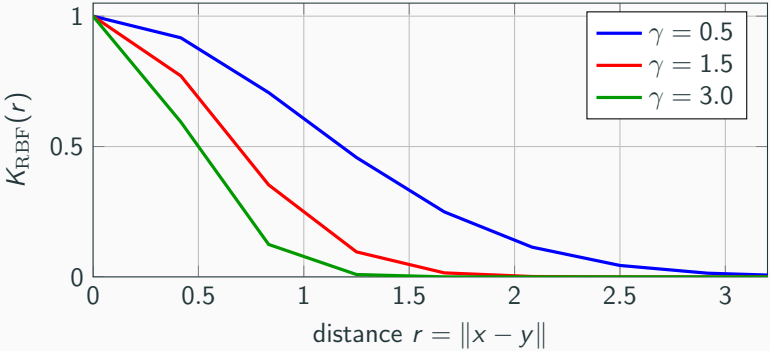
Interpretation:

- if $\|x - y\|$ is small, then $K_{\mathrm{RBF}}(x, y) \approx 1$
- if $\|x - y\|$ is large, then $K_{\mathrm{RBF}}(x, y) \approx 0$

So the RBF kernel turns Euclidean distance into a smooth similarity score. It is still a kernel method, but its notion of neighborhood is local.

How RBF γ changes locality

$$K_{\mathrm{RBF}}(r) = \exp\left(-\gamma r^2\right), \qquad r = \|x - y\|.$$



Larger γ makes similarity decay faster, so the model reacts more to nearby points than to distant ones.

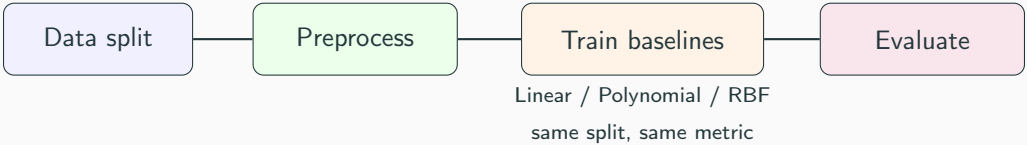
Kernel families at a glance

Kernel	Formula	Geometric meaning
Linear	$x^\top y$	raw-space inner product
Polynomial	$(\gamma x^\top y + c)^d$	interactions up to degree d
RBF	$\exp\left(-\gamma \lVert x - y \rVert^2\right)$	local similarity by distance

No kernel is universally best. A fair comparison keeps the same train/validation split, preprocessing, and evaluation metric.

`lecture_04_demo.ipynb`, Demo 5: linear vs RBF on real data.

Baseline comparison workflow



The same discipline is needed before claiming anything about a quantum feature map.

Bridge to quantum feature spaces

Classical kernels and quantum kernels

Classical kernel logic:

$$K(x, y) = \langle \phi(x), \phi(y) \rangle.$$

Quantum feature-map logic:

$$|\phi(x)\rangle = U_\phi(x) \, |0\rangle^{\otimes n}, \qquad K_Q(x, y) = |\langle \phi(x) | \phi(y) \rangle|^2.$$

The quantum kernel is a projection-based similarity: the larger the overlap between encoded states, the more similar the inputs.

One-qubit bridge example

Suppose every qubit starts in $|0\rangle$ and we encode one number by

$$|\phi(x)\rangle = R_y(x)\,|0\rangle = \cos\left(\frac{x}{2}\right)|0\rangle + \sin\left(\frac{x}{2}\right)|1\rangle\,.$$

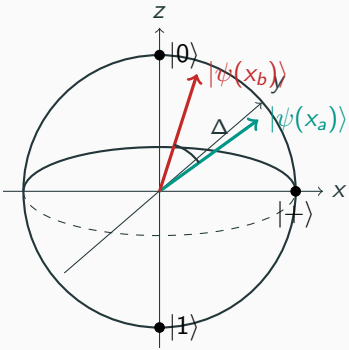
Then

$$\langle\phi(x)|\phi(y)\rangle = \cos\left(\frac{x-y}{2}\right), \qquad K_Q(x,y) = \cos^2\left(\frac{x-y}{2}\right).$$

This already behaves like a kernel: similar inputs give large overlap, separated inputs give smaller overlap.

`lecture_04_demo.ipynb`, Demo 6: one-qubit overlap preview.

Bloch-sphere geometry (one-qubit intuition)



Encoded data points become state vectors.

For pure one-qubit states, smaller angular separation gives larger overlap:

$$|\langle\psi(x_a)|\psi(x_b)\rangle|^2 = \cos^2\left(\frac{\Delta}{2}\right).$$

This is the quantum analogue of comparing feature vectors by inner products.

Bridge to Lecture 5

Lecture 5 turns this bridge into explicit encoding strategies: basis, amplitude, angle, and entangling feature maps, together with overlap-based quantum kernels.